

Álgebra Lineal Computacional

Números de Máquina

Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

2do Cuatrimestre 2026

Motivación: cálculo simbólico y numérico

Cálculo simbólico

$$x = \sqrt{2}, \quad x^2 = 2.$$

Cálculo numérico

$$x = 1.4142135623730951, \quad x^2 = 2.0000000000000004.$$

Idea central

En la computadora no trabajamos con todos los reales, sino con un conjunto finito de números representables y con reglas de redondeo.

Bases de numeración

Representación posicional

En base $b \geq 2$, un número no nulo puede escribirse como

$$(x)_b = \text{sg}(x) \tilde{a}_k \tilde{a}_{k-1} \dots \tilde{a}_0 . \tilde{a}_{-1} \tilde{a}_{-2} \dots$$

con dígitos $0 \leq \tilde{a}_i \leq b - 1$. Esta notación representa

$$x = \tilde{a}_k b^k + \tilde{a}_{k-1} b^{k-1} + \dots + \tilde{a}_0 b^0 + \tilde{a}_{-1} b^{-1} + \dots .$$

Ejemplo

- El número 517.23 en base 10 representa al número:

$$5 \cdot 10^2 + 1 \cdot 10^1 + 7 \cdot 10^0 + 2 \cdot 10^{-1} + 3 \cdot 10^{-2}$$

- El número 101.11 en base 2 representa al número:

$$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2}$$

Representación en máquina

- Se dedica porción limitada y fija en memoria a cada número.
- La cantidad de bits dedicados depende de la precisión a utilizar.
- Las computadoras representan la información en formato binario (bits)
- Estándar IEEE-754 de 1985 como convención para representar números:
 - Precisión *single*: 32 bits (4 bytes)
 - Precisión *double*: 64 bits (8 bytes)
- Surgen *errores de redondeo* al representar números reales.

Estándar IEEE-754



(A) Precisión Simple 32 bits



(B) Doble Precisión 64 bits

Representación de los números de la forma $x = \pm m \times 2^e$

IEEE-754 single precision: 32 bits

signo	exponente	mantisa
1 bit	8 bits	23 bits
s	E	M

Lectura de los campos

- $s = 0$ indica positivo y $s = 1$ indica negativo.
- El exponente guardado E no es el exponente real: se usa un **sesgo**.
- La mantisa M guarda solo los bits después del punto en $1.f$.

Para single precision

$$\text{sesgo} = 127, \quad e = E - 127.$$

Punto flotante normalizado en base 2

Forma normalizada

Todo número binario no nulo puede escribirse como

$$x = (-1)^s (1.f)_2 2^e,$$

con $s \in \{0, 1\}$, exponente $e \in \mathbb{Z}$ y una mantisa binaria f .

Observación clave

En base 2, todo número normalizado positivo empieza con 1. Ese primer 1 es redundante: IEEE-754 no lo guarda explícitamente.

Ejemplo completo: representar 13.25

Paso 1: pasar a binario y normalizar

$$13.25 = (1101.01)_2 = (1.10101)_2 2^3.$$

Paso 2: campos IEEE-754 single

- Signo: $s = 0$.
- Exponente real: $e = 3$.
- Exponente almacenado: $E = e + 127 = 130 = (10000010)_2$.
- Mantisa: se guarda lo que sigue al primer 1:

$$F = 101010000000000000000000.$$

Resultado

0 10000010 101010000000000000000000 Hex: 0x41540000.

Casos especiales en IEEE-754

Exponente	Mantisa	Interpretación
$E = 0$	$F = 0$	cero con signo: $+0$ o -0
$E = 0$	$F \neq 0$	subnormal: $(-1)^s(0.f)_2 2^{-126}$
$1 \leq E \leq 254$	cualquiera	normalizado: $(-1)^s(1.f)_2 2^{E-127}$
$E = 255$	$F = 0$	infinito: $+\infty$ o $-\infty$
$E = 255$	$F \neq 0$	NaN: resultado no numérico

Para la práctica

Los subnormales permiten representar números muy cercanos a cero, pero con menos bits efectivos de precisión.

No todos los reales son representables

Ejemplo clásico

$$0.1_{10} = 0.000110011001100110011 \dots_2$$

La expansión binaria es periódica infinita. Como solo tenemos una cantidad fija de bits, la computadora guarda un número cercano, no exactamente 0.1.

Modelo de redondeo

Para una operación aritmética $\circ \in \{+, -, \times, /\}$, se suele modelar

$$\text{fl}(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq u,$$

siempre que no haya overflow, underflow ni casos especiales.

Epsilon de máquina y unit roundoff

Dos convenciones que conviene separar

- **Machine epsilon** suele ser la distancia entre 1 y el siguiente número representable.
- **Unit roundoff** u es la cota del error relativo al redondear al más cercano.

Formato	machine epsilon	unit roundoff
float	$2^{-23} \approx 1.19 \cdot 10^{-7}$	$2^{-24} \approx 5.96 \cdot 10^{-8}$
double	$2^{-52} \approx 2.22 \cdot 10^{-16}$	$2^{-53} \approx 1.11 \cdot 10^{-16}$

Operar en máquina no es operar en $\mathbb{R}$

La computadora redondea entradas y resultados

Una suma en máquina se modela como

$$\text{fl}(\text{fl}(x) + \text{fl}(y)).$$

Aunque x e y sean representables, el resultado puede no serlo y se redondea.

Pérdida por distinta escala

Con aritmética decimal de 5 cifras:

$$x = 0.88888888 \times 10^7, \quad y = 0.1 \times 10^2.$$

$$\text{fl}(\text{fl}(x) + \text{fl}(y)) = \text{fl}(0.88888 \times 10^7 + 0.1 \times 10^2) = 0.88888 \times 10^7.$$

El sumando pequeño desaparece porque queda fuera de las 5 cifras retenidas.

Error absoluto y error relativo

Definiciones

Si $\hat{x}$ aproxima a x , definimos

$$\text{error absoluto} = |x - \hat{x}|, \quad \text{error relativo} = \frac{|x - \hat{x}|}{|x|} \quad (x \neq 0).$$

Por qué importa el error relativo

Un error absoluto de 10^{-8} puede ser despreciable si $x \approx 1$, pero enorme si el resultado esperado es $x \approx 10^{-10}$.

En álgebra lineal computacional

Nos interesa cómo los errores de redondeo se propagan en algoritmos: sumas, productos internos, resolución de sistemas y factorizaciones.

Cancelación: el problema

Qué ocurre

La cancelación aparece cuando restamos dos cantidades muy cercanas:

$$a - b, \quad a \approx b.$$

Los dígitos más significativos se cancelan y el resultado queda determinado por dígitos menos confiables.

Ejemplo decimal con 5 cifras

$$a = 1.234567, \quad b = 1.234321, \quad a - b = 0.000246.$$

Si antes redondeamos:

$$\text{fl}(a) = 1.2346, \quad \text{fl}(b) = 1.2343, \quad \text{fl}(a) - \text{fl}(b) = 0.0003.$$

El error relativo en la resta es aproximadamente 22 %.

Cancelación catastrófica: idea clave

No toda cancelación es igual de grave

La resta de números cercanos puede ser exacta en máquina bajo ciertas condiciones. El problema catastrófico aparece cuando la resta expone errores de redondeo que ya estaban en los operandos.

Regla práctica

Si una fórmula contiene una resta de dos cantidades casi iguales, conviene buscar una forma algebraicamente equivalente que evite esa resta.

Ejemplo: $\frac{1-\cos(x)}{x^2}$

Dos fórmulas matemáticamente equivalentes

Usando la identidad $1 - \cos(x) = 2 \sin^2(x/2)$,

$$\frac{1 - \cos(x)}{x^2} = \frac{2 \sin^2(x/2)}{x^2}.$$

Ambas tienen límite $1/2$ cuando $x \rightarrow 0$.

Pero numéricamente no se comportan igual

Para x pequeño, $\cos(x) \approx 1$. Entonces $1 - \cos(x)$ resta dos números casi iguales. En double, para algunos x chicos, $\cos(x)$ se redondea directamente a 1 y el numerador queda 0.

Ejemplo numérico de cancelación

x	$\frac{1 - \cos(x)}{x^2}$	$\frac{2 \sin^2(x/2)}{x^2}$
10^{-4}	0.4999999996961	0.4999999999583
10^{-6}	0.500044450291	0.4999999999999
10^{-8}	0	0.5
10^{-9}	0	0.5

Lectura

La primera fórmula es matemáticamente correcta, pero numéricamente inestable cerca de cero. La segunda evita la resta problemática y conserva precisión.

Cancelación Catastrófica

$$\frac{1 - \cos(x)}{x^2} \text{ vs. } \frac{2 \cdot \sin^2\left(\frac{x}{2}\right)}{x^2}, \quad -4 \times 10^{-8} \leq x \leq 4 \times 10^{-8}$$

